

Stochastic simulation

Alessandro Cavalieri

2026-02-20

Table of contents

1	Inversion sampling	1
2	Rejection sampling	1
2.1	Example	2
3	Interactive rejection sampling	4

1 Inversion sampling

Inversion sampling is a method for generating random samples from a target distribution $p(x)$ by using its cumulative distribution function (CDF) $F(x)$. The algorithm works as follows:

1. Compute the CDF $F(x) = \int_{-\infty}^x p(t)dt$.
2. Generate a uniform random number u from the interval $[0, 1]$.
3. Find the value x such that $F(x) = u$. This can be done by computing the inverse CDF, denoted as $F^{-1}(u)$, which gives us the sample from the target distribution.

2 Rejection sampling

Rejection sampling is a method for generating random samples from a target distribution $p(x)$ by using a proposal distribution $q(x)$ that is easy to sample from. The algorithm works as follows:

1. Choose a proposal distribution $q(x)$ and a constant M such that $Mq(x) \geq p(x)$ for all x .
2. Repeat the following steps until you have the desired number of samples:
 - a. Sample x' from the proposal distribution $q(x)$.

- b. Generate a uniform random number u from the interval $[0, 1]$.
- c. If $u < \frac{p(x')}{Mq(x')}$, accept x' as a sample from $p(x)$; otherwise, reject it and repeat the process.

Theoretical Acceptance rate

$$\mathbb{E}\left[U < \frac{p(X)}{Mq(X)} \mid X\right] = \int \frac{p(x)}{Mq(x)} q(x) dx = \frac{1}{M} \overbrace{\int p(x) dx}^{=1} = \frac{1}{M}$$

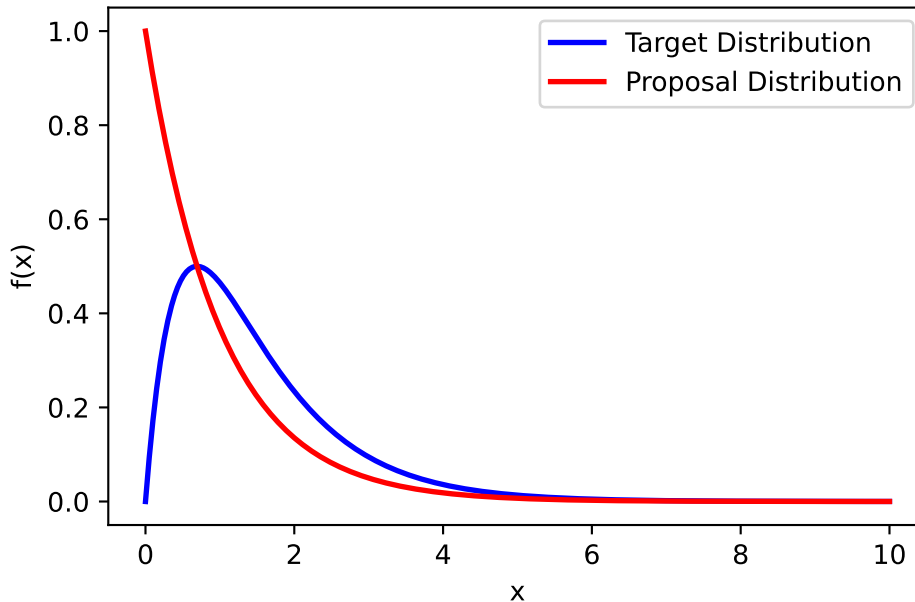
The efficiency of rejection sampling depends on how well the proposal distribution $q(x)$ approximates the target distribution $p(x)$. If M is large, the acceptance rate will be low, leading to many rejected samples. Therefore, it is crucial to choose a proposal distribution that closely matches the shape of the target distribution to improve efficiency.

2.1 Example

Suppose you want to sample from

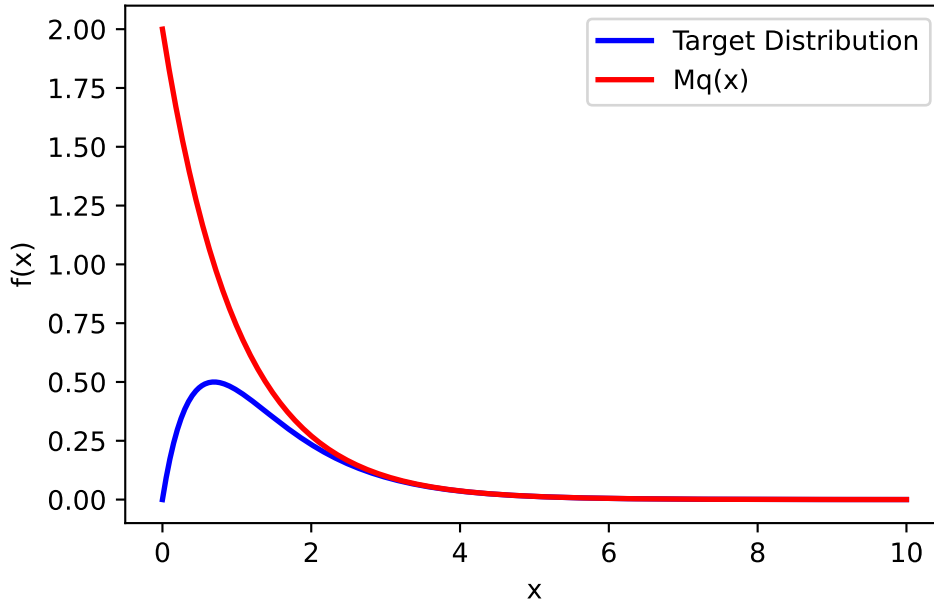
$$p(x) = \begin{cases} 2e^{-x}(1 - e^{-x}) & \text{if } x \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

having $q(x) = e^{-x}$ for $x \geq 0$ as proposal distribution.

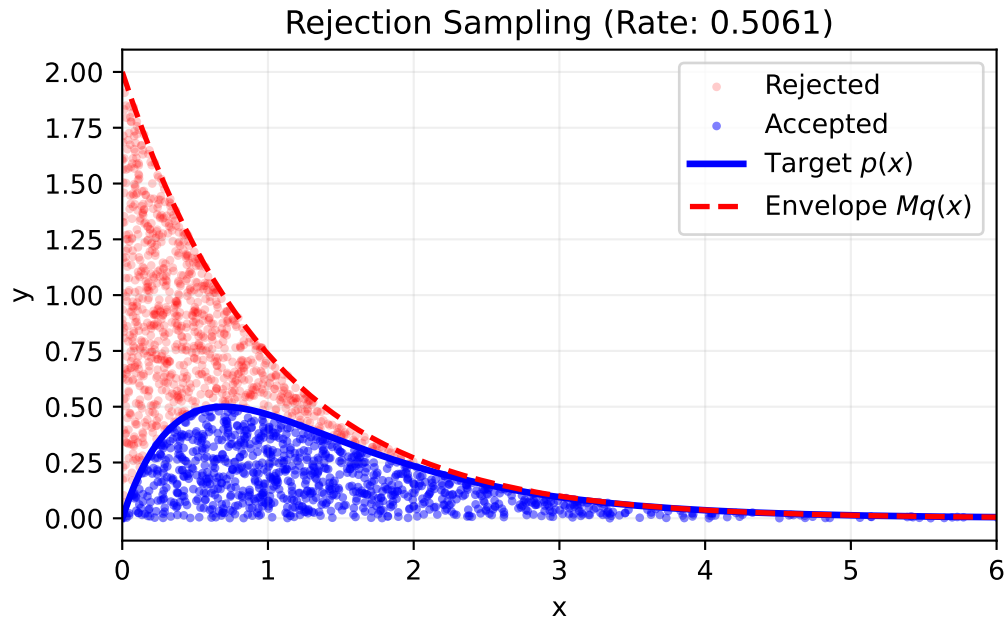


We have to choose a constant M such that $Mq(x) \geq p(x)$ for all x :

$$M = \max_{x \geq 0} \frac{p(x)}{q(x)} = \max_{x \geq 0} \frac{2e^{-x}(1 - e^{-x})}{e^{-x}} = \max_{x \geq 0} 2(1 - e^{-x}) = 2$$



We can now try to run point 2b - 2c until we reach 1000 accepted samples:



We can see that the empirical acceptance rate is close to the theoretical one, given by

$$\text{Acceptance Rate} = \frac{1}{M} = \frac{1}{2} = 0.5$$

3 Interactive rejection sampling

```
///| panel: input
viewof lambda = Inputs.range([0.1, 3], {step: 0.1, label: " (Proposal)", value: 1})
viewof M = Inputs.range([1, 10], {step: 0.1, label: "M (Scaling)", value: 3})
viewof run = Inputs.button("Generate 500 Samples")

// 1. The Simulation Logic
// This runs whenever the 'run' button is clicked
results = {
  run; // Reference 'run' to trigger recalculation
  const n = 500;
  const accepted = [];
  const rejected = [];
  let attempts = 0;

  while (accepted.length < n && attempts < 5000) {
    attempts++;
    // Sample x from exponential proposal: x = -ln(1-u)/lambda
    const x_prime = -Math.log(1 - Math.random()) / lambda;
    const u = Math.random();

    const p_val = x_prime >= 0 ? 2 * Math.exp(-x_prime) * (1 - Math.exp(-x_prime)) : 0;
    const mq_val = M * (lambda * Math.exp(-lambda * x_prime));

    const point = {x: x_prime, y: u * mq_val};

    if (u < p_val / mq_val) {
      accepted.push({...point, status: "Accepted"});
    } else {
      rejected.push({...point, status: "Rejected"});
    }
  }
  return {accepted, rejected, rate: accepted.length / attempts};
}
```

```

// 2. Display Statistics
md`**Acceptance Rate:** ${results.rate * 100}.toFixed(2)}% | **Total Iterations:** ${results

// 3. The Visualization
Plot.plot({
  width: 700,
  height: 400,
  grid: true,
  y: {domain: [0, 2], label: "Density"},
  x: {domain: [0, 6]},
  color: {
    domain: ["Target p(x)", "Envelope M*q(x)", "Accepted", "Rejected"],
    range: ["blue", "green", "blue", "#ff000044"]
  },
  marks: [
    // Static Lines
    Plot.line(Array.from({length: 300}, (_, i) => i * 0.02), {
      x: d => d,
      y: d => 2 * Math.exp(-d) * (1 - Math.exp(-d)),
      stroke: () => "Target p(x)", strokeWidth: 2
    }),
    Plot.line(Array.from({length: 300}, (_, i) => i * 0.02), {
      x: d => d,
      y: d => M * (lambda * Math.exp(-lambda * d)),
      stroke: () => "Envelope M*q(x)", strokeWidth: 2, stroke: "#ff0000"
    }),
    // Sampled Points
    Plot.dot(results.accepted, {x: "x", y: "y", fill: "blue", r: 2, title: "Accepted"}),
    Plot.dot(results.rejected, {x: "x", y: "y", fill: "#ff0000", r: 1.5, fillOpacity: 0.2, title: "Rejected"}),
    Plot.ruleY([0])
  ]
})

```